



Discovery Piscine

Module3 - Python

*Հակիրճ. Այս Module3-ում մենք կտեսնենք, թե ինչպես օգտագործել
ցիկլերը:*

Contents

1	Մի քանի խոսք այս Discovery Pool-ի մասին	2
2	Ներածություն	4
3	Ընդհանուր հրահանգներ	5
4	Վարժություն 00: Մինչև 25	6
5	Վարժություն 01: multiplication_table	8
6	Վարժություն 02: i_got_that	9
7	Վարժություն 03: advanced_mult	10
8	Հանձնում և peer-evaluation	11

Chapter 1

Մի քանի խոսք այս Discovery Pool-ի մասին

Բարի գալուստ:

Դուք կսկսեք համակարգչային ծրագրավորման այս Discovery Piscine-ի Module-ներից մեկը: Մեր նպատակն է ձեզ ծանոթացնել այն կողի հետ, որը թաքնված է ձեր ամեն օր օգտագործած ծրագրերի հետևում, և ընկղմել ձեզ peer learning-ի՝ 42-ի կրթական մոդելի մեջ:

Ծրագրավորումը տրամաբանության մասին է, ոչ թե մաթեմատիկայի: Այն ձեզ տալիս է հիմնական կառուցվածքային բլոկներ, որոնք դուք կարող եք հավաքել անթիվ եղանակներով: Խնդրի համար չկա մեկ «ճիշտ» լուծում. ձեր լուծումը կլինի յուրահատուկ, ինչպես որ ձեր ընկերներից յուրաքանչյուրի լուծումը կլինի:

Արագ թե դանդաղ, նրբագեղ թե խառնաշփոթ, քանի դեռ այն աշխատում է՝ դա է կարևորը: Այս կառուցվածքային բլոկները կկազմեն հրահանգների հաջորդականություն (հաշվարկների, ցուցադրումների և այլնի համար), որը համակարգիչը կկատարի ձեր նախագծած հերթականությամբ:

Փոխանակ ձեզ տրամադրելու դասընթաց, որտեղ յուրաքանչյուր խնդիր ունի միայն մեկ լուծում, մենք ձեզ տեղավորում ենք peer-learning միջավայրում: Դուք կփնտրեք տարրեր, որոնք կարող են օգնել ձեզ դիմակայել ձեր մարտահրավերի կկատարելագործեք դրանք թեստավորման ու փորձարկման միջոցով և ի վերջո կստեղծեք ձեր սեփական ծրագիրը: Քննարկեք ուրիշների հետ, կիսվեք ձեր տեսակետներով, միասին նոր գաղափարներ ծնեք և ամեն ինչ ինքներդ թեստավորեք՝ համոզվելու համար, որ այն աշխատում է:

Peer evaluation-ը կարևոր հնարավորություն է՝ բացահայտելու այլընտրանքային մոտեցումներ և ձեր ծրագրում նկատելու հնարավոր խնդիրներ, որոնք դուք գուցե բաց եք թողել (մտածեք, թե որքան հիասթափեցնող կարող է լինել ծրագրի խափանումը): Յուրաքանչյուր գնահատող ձեր աշխատանքին կմոտենա տարբեր կերպ՝ ինչպես տարբեր սպասելիքներ ունեցող հաճախորդներ՝ ձեզ տալով նոր տեսանկյուններ: Դուք նույնիսկ կարող եք կապեր ստեղծել ապագա համագործակցությունների համար:

Այս Piscine-ի ավարտին ձեր ճանապարհորդությունը կլինի յուրահատուկ: Դուք կդիմակայեք տարբեր մարտահրավերների, կվավերացնեք տարբեր նախագծեր և կընտրեք ուրիշներից տարբեր ուղիներ, և դա միանգամայն նորմալ է: Սա և՛ կոլեկտիվ, և՛ անհատական փորձառություն է, և յուրաքանչյուրը դրանից ինչ-որ բան կստանա:

Հաջողություն բոլորին. հուսով եմք՝ դուք կվայելեք հայտնագործման այս ճանապարհորդությունը:

Chapter 2

Ներածություն

Ինչ ձեզ ցույց կտա այս Module-ը՝

- Դուք կսովորեք, թե ինչպես կատարել որոշ ցիկլեր:

Chapter 3

Ընդհանուր հրահանգներ

Եթե այլ բան նշված չէ, հետևյալ կանոնները կիրառվում են այս Piscine-ի ամեն օր:

- Այս փաստաթուղթը միակ վստահելի աղբյուրն է: Մի ապավինեք լուրերին:
- Այս փաստաթուղթը կարող է թարմացվել մինչև հանձնման վերջնաժամկետից մեկ ժամ առաջ:
- Առաջադրանքները պետք է կատարվեն նշված հերթականությամբ: Հետագա առաջադրանքները չեն գնահատվի, եթե բոլոր նախորդները ճիշտ չեն կատարվել:
- Ուշադիր եղեք ձեր ֆայլերի և թղթապանակների մուտքի իրավունքների նկատմամբ:
- Ձեր առաջադրանքները կգնահատվեն ձեր Piscine-ի ընկերների կողմից:
- Բոլոր shell առաջադրանքները պետք է աշխատեն /bin/bash-ի միջոցով:
- Դուք չպետք է ձեր հանձնման աշխատանքային տարածքում թողնեք որևէ ֆայլ, բացի նրանցից, որոնք հստակ պահանջվում են առաջադրանքներում:
- Հարց ունե՞ք: Հարցրեք ձեր ձախ կողմի հարևանին: Եթե ոչ, փորձեք ձեր աջ կողմի հարևանին:
- Ձեզ անհրաժեշտ ամեն տեխնիկական պատասխան կարող եք գտնել man էջերում կամ առցանց:
- Հիշեք օգտագործել ձեր intranet-ի Piscine ֆորումը և Slack-ը:
- Օրինակները մանրակրկիտ կարդացեք, քանի որ դրանք կարող են բացահայտել պահանջներ, որոնք միանգամից ակնհայտ չեն առաջադրանքի նկարագրության մեջ:
- Ի սեր Թորի, ի սեր Օդինի: Օգտագործե՛ք ձեր ուղեղը:

Chapter 4

Վարժություն 00: Մինչև 25

	Առաջադրանք 00
Բարձրանանք մինչև 25	
Հանձնման պահոց՝ <code>ex00/</code>	
Հանձնվելիք ֆայլեր՝ <code>to25.py</code>	
Թույլատրված ֆունկցիաներ՝ <code>All</code>	

- Ստեղծեք ծրագիր՝ `to25.py` անունով:
- Համոզվեք, որ ծրագիրը գործարկելի է:
- Ծրագիրը պետք է՝
 - Ընդունի օգտատիրոջ մուտքագրումը, որը կպահվի թվային փոփոխականում:
 - Օգտագործի `while` մուտքագրված թվից մինչև 25 բոլոր թվերը ցուցադրելու համար:
 - Եթե մուտքագրված թիվը մեծ է 25-ից, ցուցադրի `"Error"`՝ նոր տողով ուղեկցված:

```
?> ./to25.py
Enter a number less than 25
45
Error
?> ./to25.py
Enter a number less than 25
20
Inside the loop, my variable is 20
Inside the loop, my variable is 21
Inside the loop, my variable is 22
Inside the loop, my variable is 23
Inside the loop, my variable is 24
Inside the loop, my variable is 25
?>
```



Օգտագործեք while ցիկլ:

Chapter 5

Վարժություն 01: multiplication_table

	Առաջադրանք 01
Բազմապատկման աղյուսակների վերադարձը	
Հանձնման պահոց՝ <i>ex01/</i>	
Հանձնվելիք ֆայլեր՝ <i>multiplication_table.py</i>	
Թույլատրված ֆունկցիաներ՝ <i>All</i>	

- Ստեղծել ծրագիր՝ *multiplication_table.py* անունով:
- Համոզվել, որ ծրագիրը գործարկելի է:
- Ծրագիրը պետք է՝
 - Ընդունի օգտատիրոջ մուտքագրումը, որը կպահվի թվային փոփոխական
 - Ցուցադրի այդ թվի բազմապատկման աղյուսակը (օրինակ՝ եթե մուտքագրումը 2 է, ցուցադրի 2-ի բազմապատկման աղյուսակը):

```
?> ./multiplication\_table.py
Enter a number
8
0 x 8 = 0
1 x 8 = 8
2 x 8 = 16
3 x 8 = 24
4 x 8 = 32
5 x 8 = 40
6 x 8 = 48
7 x 8 = 56
8 x 8 = 64
9 x 8 = 72
?>
```

Chapter 6

Վարժություն 02: i_got_that

	Առաջադրանք 02
Հասկացա՞ր	
Հանձնման պահոց՝ <i>ex02/</i>	
Հանձնվելիք ֆայլեր՝ <i>i_got_that.py</i>	
Թույլատրված ֆունկցիաներ՝ All	

- Ստեղծեք ծրագիր՝ *i_got_that.py* անունով:
- Համոզվեք, որ ծրագիրը գործարկելի է:
- Ծրագիրը պետք է՝
 - Օգտագործի *while* ցիկլ, որը շարունակաբար ընդունում է օգտատիրոջ մուտքագրումը և պատասխանում "I got that! Anything else?"՝ յուրաքանչյուր մուտքագրումից հետո:
 - Ցիկլը պետք է դադարի միայն այն ժամանակ, երբ օգտատերը մուտքագրում է "STOP":

```
?> ./i_got_that.py
What you gotta say? : Hello
I got that! Anything else? : I like ponies
I got that! Anything else? : stop...
I got that! Anything else? : STOP
?>
```



Ուսումնասիրեք 'while'-ը և 'break'-ը:

Chapter 7

Վարժություն 03: advanced_mult

	Առաջադրանք 03
Բազմապատկման աղյուսակների վերադարձի վերադարձը	
Հանձնման պահոց՝ <i>ex03/</i>	
Հանձնվելիք ֆայլեր՝ <code>advanced_mult.py</code>	
Թույլատրված ֆունկցիաներ՝ <code>All</code>	

- Ստեղծեք ծրագիր՝ `advanced_mult.py` անունով:
- Համոզվեք, որ ծրագիրը գործարկելի է:
- Ծրագիրը պետք է՝
 - Ցուցադրի 0-ից 10 բոլոր բազմապատկման աղյուսակները հետևյալ ձևաչափով՝

```
?> ./advanced_mult.py
Table of 0: 0 0 0 0 0 0 0 0 0 0 0
Table of 1: 0 1 2 3 4 5 6 7 8 9 10
Table of 2: 0 2 4 6 8 10 12 14 16 18 20
Table of 3: 0 3 6 9 12 15 18 21 24 27 30
Table of 4: 0 4 8 12 16 20 24 28 32 36 40
Table of 5: 0 5 10 15 20 25 30 35 40 45 50
Table of 6: 0 6 12 18 24 30 36 42 48 54 60
Table of 7: 0 7 14 21 28 35 42 49 56 63 70
Table of 8: 0 8 16 24 32 40 48 56 64 72 80
Table of 9: 0 9 18 27 36 45 54 63 72 81 90
Table of 10: 0 10 20 30 40 50 60 70 80 90 100
?>
```

- Դուք իրավունք ունեք օգտագործել միայն երկու `while` ցիկլ:

Chapter 8

Հանձնում և peer-evaluation

- Ձեր home directory-ի արմատում պետք է ունենաք discovery_piscine թղթապանակը:
- discovery_piscine թղթապանակի ներսում պետք է ունենաք թղթապանակ՝ module3 անունով:
- module3 թղթապանակի ներսում պետք է ունենաք թղթապանակ յուրաքանչյուր վարժության համար:
- Վարժություն 00-ը պետք է լինի ex00 թղթապանակում, Վարժություն 01-ը՝ ex01 թղթապանակում, և այլն:
- Յուրաքանչյուր վարժության թղթապանակ պետք է պարունակի առաջադրանք պահանջված ֆայլերը:



Խնդրում ենք նկատի ունենալ, որ ձեր defense-ի ընթացքում այն ամենը, ինչ առկա չէ module-ի թղթապանակում, չի ստուգվի: